MOVZZ — Master Roadmap (Combined)

> Reliability-Orchestrated Mobility Platform · Chennai, India Last updated: March 2026

Legend

Symbol	Meaning
☑ DONE	Implemented and working
🔴 CRITICAL	Security risk / launch blocker
🟠 HIGH	Needed for beta users
🟡 MEDIUM	Important but deferrable post-launch
🟢 NICE	Post-launch enhancement
☐ TODO	Not yet started
🤖 AI	AI engine task

Grand Summary

Category	Total	Done	Remaining
Section 0 — Security Hardening	11	0	11
Section 1 — Foundation	33	33	0
Section 2 — Platform Features	26	9	17
Section 3 — AI Intelligence Layer	23	8	15
Section 4 — Production & Deployment	14	0	14
TOTAL	**107**	**50**	**57**

Progress: 50/107 tasks (47%)

SECTION 0 — Security Hardening ☐ (0/11 — All Critical, Do First)

> Confirmed in live code audit (March 9, 2026). App is insecure until S1–S4 are fixed. Do these before any real users or production deployment.

#	Priority	Task	File(s)
S1	🔴	**Remove hardcoded JWT fallback secret** — `jwt.service.ts` uses `|| 'fallback-secret-change-in-production'`; anyone who reads source can forge valid tokens. Throw `Error('JWT_SECRET is required')` on startup instead	`backend/src/services/jwt.service.ts`

#	Priority	Task	File(s)
S2	⬤	**Fix auth race condition** — `auth.controller.ts` uses `findFirst + create`; two concurrent sign-ups with same phone crash DB. Replace with `prisma.user.upsert({ where: { phone }, update: {}, create: {...} })`	`backend/src/controllers/auth.controller.ts`
S3	⬤	**Fix OAuth race condition** — `oauth.controller.ts` has same `findFirst + create` pattern (comment says "upsert" but code does not). Same upsert fix	`backend/src/controllers/oauth.controller.ts`
S4	⬤	**Add admin role protection** — `User` model has no `role` field; all `/admin/` routes only check `authMiddleware`, meaning any logged-in user can access all bookings, provider data, analytics. Add `role String @default("user")` to schema + create `adminMiddleware` + protect all admin routes	`schema.prisma`, new `backend/src/middleware/admin.middleware.ts`, `backend/src/routes/admin.routes.ts`
S5	⬤	**Fix hardcoded API URL** — `api/client.ts` has `baseURL: 'http://localhost:3000/api/v1'` — won't work in any deployed environment. Replace with `import.meta.env.VITE_API_URL` fallback to localhost. Same fix in `App.jsx` and `Admin.jsx`	`frontend/src/api/client.ts`, `frontend/src/App.jsx`, `frontend/src/Admin.jsx`
S6	⬤	**Protect env files** — `frontend/.env` exists on disk; `frontend/.env.example` does not exist. Verify `.env` is in `.gitignore`; create `.env.example` with placeholder values for all keys	`.gitignore`, `frontend/.env.example` NEW, `backend/.env.example` NEW
S7	◍	**OTP rate limiting** — no rate limit on `/verify-otp`; attacker can brute-force all 1M 6-digit codes. Add `express-rate-limit`: 5 attempts per phone per 10 min	`backend/src/routes/auth.routes.ts`, new `backend/src/middleware/otp-limiter.ts`
S8	◍	**Fix Socket memory leak** — `bookingStore.ts` adds new listeners on every navigation without removing old ones. Store handler refs in a `Map`; use exact same refs in `socket.off()`	`frontend/src/stores/bookingStore.ts`

#	Priority	Task	File(s)
S9	⬤	**Fix dual fare engine** — `quotes.service.ts` has its own `calculateFare()` that produces different prices than `fare.service.ts`. User sees ₹150, gets charged ₹180. Delete local method; import and call `estimateFares()` from `fare.service.ts`	`backend/src/services/quotes.service.ts`
S10	⬤	**Add token expiry** — JWT stored in localStorage with no expiry tracking. Add `tokenExpiry = Date.now() + 7d` on login; check on app startup; auto-logout if expired	`frontend/src/stores/authStore.ts`
S11	⬤	**Harden CORS + global rate limit** — lock CORS `origin` to production domains only; add global rate limiter (100 req / 15 min / IP); confirm Helmet headers active	`backend/src/index.ts`

SECTION 1 — Foundation ☑ (33/33 Complete)

Phase 1 — Core Backend ☑ (12/12)

#	Task	Status	Key File(s)
1	Express + TypeScript + Prisma scaffold	☑ DONE	`backend/src/index.ts`
2	PostgreSQL schema — Users, Providers, Bookings, BookingAttempts, BookingLogs	☑ DONE	`backend/prisma/schema.prisma`
3	Redis cache with in-memory fallback	☑ DONE	`backend/src/config/redis.ts`
4	OTP authentication (phone) with JWT	☑ DONE	`backend/src/controllers/auth.controller.ts`
5	Fare engine — mode-specific rates, surge, Haversine×1.35, airport detection, paise precision	☑ DONE	`backend/src/services/fare.service.ts`
6	Provider scoring — reliability from historical success rate	☑ DONE	`backend/src/services/provider-scoring.service.ts`
7	Booking state machine — SEARCHING → CONFIRMED → IN_PROGRESS → COMPLETED / FAILED / CANCELLED / MANUAL_ESCALATION	☑ DONE	`backend/src/services/booking.service.ts`
8	Recovery service — 3-level retry, auto-escalation, ₹100 compensation credit	☑ DONE	`backend/src/services/recovery.service.ts`
9	Quotes API — ranked options with reliability rationale	☑ DONE	`backend/src/services/quotes.service.ts`

#	Task	Status	Key File(s)
10	Booking API — create, poll status, cancel	☑ DONE	`backend/src/controllers/booking.controller.ts`
11	Admin dashboard API — stats, provider list	☑ DONE	`backend/src/controllers/admin.controller.ts`
12	Rate limiting, CORS, Helmet security headers	☑ DONE	`backend/src/index.ts`

Phase 2 — Frontend Prototype ☑ (7/7)

#	Task	Status	Key File(s)
13	React + Vite scaffold with Zustand state management	☑ DONE	`frontend/src/App.jsx`
14	5-screen flow: Landing → Auth → Transport → Destination → Results	☑ DONE	`frontend/src/App.jsx`
15	Transport mode selector — CAB (Economy/Comfort/Premium), Bike, Auto, Metro	☑ DONE	`frontend/src/App.jsx`
16	Results screen — ranked quote cards with reliability score, ETA, price, tags	☑ DONE	`frontend/src/App.jsx`
17	Auth store — OTP send/verify, JWT storage, auth state	☑ DONE	`frontend/src/stores/authStore.ts`
18	Booking store — fetchQuotes, createBooking, Socket.IO state sync	☑ DONE	`frontend/src/stores/bookingStore.ts`
19	API client (axios) with JWT header injection	☑ DONE	`frontend/src/api/client.ts`

Phase 3 — Production Hardening ☑ (5/5)

#	Task	Status	Notes
20	WebSockets (Socket.IO) — `booking:state_changed` server-push, per-user rooms, JWT on handshake	☑ DONE	`backend/src/config/socket.ts`
21	Google OAuth2 — direct REST flow, no Passport.js	☑ DONE	`backend/src/controllers/oauth.controller.ts`
22	Mapbox geocoding autocomplete — 300ms debounce, India filter	☑ DONE	`frontend/src/App.jsx`
23	Interactive map — react-map-gl, green/orange markers, auto-pan	☑ DONE	`frontend/src/App.jsx`
24	Chennai preset location chips with real lat/lng	☑ DONE	`frontend/src/App.jsx`

Phase 4 — Background Jobs / BullMQ ☑ (4/4)

#	Task	Status	Notes

#	Task	Status	Notes
25	BullMQ queue singletons — booking-timeout, recovery-retry, sms-dispatch	☑ DONE	`backend/src/config/queues.ts`
26	Booking timeout worker — auto-cancel SEARCHING after 5 min	☑ DONE	`backend/src/workers/booking-timeout.worker.ts`
27	Recovery retry worker — async provider retry, 2s delay	☑ DONE	`backend/src/workers/recovery.worker.ts`
28	SMS dispatch worker — 3 retries, exponential backoff, Twilio-ready (mock in dev)	☑ DONE	`backend/src/workers/sms.worker.ts`

Phase 5 — Payments ☑ (4/4)

#	Task	Status	Notes
29	Razorpay — create payment link endpoint	☑ DONE	`POST /api/v1/payments/create-link`
30	Razorpay — verify payment + HMAC-SHA256 signature check	☑ DONE	`POST /api/v1/payments/verify`
31	Frontend — Razorpay Payment Link redirect + return detection	☑ DONE	`frontend/src/App.jsx`
32	Provider payout tracking (T+2 terms)	☑ DONE	`Payout` table, `payment.service.ts`

Phase 9 — Admin Panel ☑ (5/5)

#	Task	Status	Notes
33	Full Admin Panel UI at `/admin` — 4 tabs: Dashboard, Escalations, Providers, Metrics	☑ DONE	`frontend/src/Admin.jsx`
34	Live booking map — react-map-gl, color-coded markers by state	☑ DONE	`frontend/src/Admin.jsx`
35	Provider management — list, pause/resume, add new provider	☑ DONE	`frontend/src/Admin.jsx`
36	Manual escalation queue — ops confirms MANUAL_ESCALATION bookings	☑ DONE	`frontend/src/Admin.jsx`
37	Analytics tab — today's booking states, weekly revenue, top providers	☑ DONE	`frontend/src/Admin.jsx`

Phase 7 — Infrastructure (partial) ☑ (3/4)

#	Task	Status	Notes
38	Sentry error tracking — `@sentry/node` + `@sentry/react`, gated on env var	☑ DONE	`backend/src/index.ts`, `frontend/src/main.jsx`
39	Neon.tech — cloud Postgres DATABASE_URL swap ready in `backend/.env`	☑ DONE	`backend/.env` (commented format)

#	Task	Status	Notes
40	AWS S3 — presigned URL pattern, profile photo upload endpoint	☑ DONE	`backend/src/services/s3.service.ts`

Phase 6 — Notifications (partial) ☑ (1/4)

#	Task	Status	Notes
41	Transactional email via Resend — confirmation, cancellation, compensation	☑ DONE	`backend/src/services/email.service.ts`

Phase 10 — Mobile (partial) ☑ (1/6)

#	Task	Status	Notes
42	PWA — vite-plugin-pwa, service worker, web manifest, offline cache, installable	☑ DONE	`frontend/vite.config.js`

SECTION 2 — Platform Features ☐ (9/26 Done, 17 Remaining)

Phase 6 — Notifications (remaining) 🔄 (1/4 done)

#	Priority	Task	Notes
43	🟤	**Twilio SMS — real OTP delivery** — replace `console.log` mock in `sms.worker.ts` with Twilio client; verify India DLT sender ID	Needs `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_PHONE_NUMBER`
44	🟤	FCM push notifications — booking state changes, driver ETA	Needs React Native or PWA service worker + Firebase project
45	⚪	WhatsApp API via Twilio — booking alerts, driver confirmation	Same Twilio credentials, different endpoint; create message templates

Phase 7 — Infrastructure (remaining) 🔄 (3/4 done)

#	Priority	Task	Notes
46	⚪	CDN — CloudFront or Cloudflare in front of S3	Quick win ~1 hr after S3 verified

Phase 8 — Provider Integrations ☐ (0/6 — all new)

#	Priority	Task	File(s)
P0	🟤	**Create `IProvider` interface + `ProviderRegistry`** — standard contract for all providers: `getEstimate()`, `checkAvailability()`, `bookRide?()`	`backend/src/services/providers/provider.interface.ts` NEW, `provider.registry.ts` NEW

#	Priority	Task	File(s)
47	⬤	**Fast Track Cabs integration** — WhatsApp Business API booking or manual confirmation flow; show price with "confirmation required" tag	`backend/src/services/providers/fasttrack.provider.ts` NEW
48	⬤	**Chennai Call Taxi integration** — same pattern as Fast Track	`backend/src/services/providers/chennai-call-taxi.provider.ts` NEW
49	⬤	Uber/Ola/Rapido aggregator fallback — reverse-engineer price estimate APIs (no booking); Redis cache 5 min; graceful fallback if unavailable	`uber.provider.ts`, `ola.provider.ts`, `rapido.provider.ts` NEW
50	⬤	Provider onboarding portal — KYC + doc upload UI; `s3.providerDocKey()` already wired	Admin UI new tab + upload flow
P1	⬤	**Provider Health Dashboard** — new Admin tab: status, success rate, response time, rate limits per provider	`frontend/src/Admin.jsx`, `GET /api/v1/admin/providers/health` NEW

Phase 10 — Mobile Apps (remaining) 🔄 (1/6 done)

#	Priority	Task	Notes
51	⬤	React Native user app — port 5-screen flow (Expo recommended)	Reuse existing API + WebSocket
52	⬤	React Native driver app — accept/reject rides, navigation	Separate app; driver-specific Socket.IO events
53	⬤	Driver location tracking — real-time GPS `driver:location_update` event	New event in `backend/src/config/socket.ts`
54	⬤	In-app FCM push notifications	Firebase + React Native Push Notification library
55	⬤	App Store + Play Store deployment	Expo EAS build

SECTION 3 — AI Intelligence Layer 🤖 (8/23 Done, 15 Remaining)

> **Achieved targets (post AI Week 1):** Booking success rate 75% → tracked. Confirmation time 45s → tracked.
> **Remaining target:** Revenue +10–15% via fair dynamic pricing (AI Week 3).

AI Week 1 — Core AI Foundation ☑ (8/8)

#	Task	Status	Files

#	Task	Status	Files
AI-1	TypeScript AI interfaces — `RideContext`, `ProviderMetrics`, `ReliabilityScore`, `ScoreBreakdown`, all enums	☑ DONE	`backend/src/types/ai.types.ts`
AI-2	Context Builder Service — Chennai zone detection, Haversine distance, peak hours, weekend, ride type	☑ DONE	`backend/src/services/context-builder.service.ts`
AI-3	Provider Metrics Service — hourly rates, recency windows (1h/6h/24h/7d), streaks, Redis 5-min cache	☑ DONE	`backend/src/services/provider-metrics.service.ts`
AI-4	Reliability Predictor — `predictReliability()` with 8 contextual adjustments	☑ DONE	`backend/src/services/provider-scoring.service.ts`
AI-5	Orchestration Service — `decideStrategy()` + `executeStrategy()` with SEQUENTIAL / PARALLEL_2 / PARALLEL_3 / CASCADE / EMERGENCY	☑ DONE	`backend/src/services/ai/orchestration.service.ts`
AI-6	Failure Detector Service — risk scoring, tiered interventions, ₹50 credit at riskScore ≥ 85	☑ DONE	`backend/src/services/ai/failure-detector.service.ts`
AI-7	Prisma migration — AI fields on Booking + Provider	☑ DONE	`backend/prisma/schema.prisma`
AI-8	Wire AI into booking.service.ts — `assignWithAI()` with fallback to legacy `assignProvider()`	☑ DONE	`backend/src/services/booking.service.ts`

AI Week 2 — Performance + Visible AI in UI ☐ (0/5)

#	Priority	Task	Files
AI-9	◍	**Nightly Aggregation Job** — BullMQ cron 2 AM; per-provider success rates by hour/day/zone/distance; stores in `ProviderMetricsCache` table + updates Redis	`backend/src/jobs/nightly-aggregation.job.ts` NEW
AI-10	◍	**Cache Service** — `getOrCompute(key, ttl, fn)` pattern; key prefixes: `provider:{id}:metrics` (5 min), `context:weather:{lat},{lng}` (30 min), `context:traffic` (5 min), `zone:mapping` (1 day)	`backend/src/services/cache.service.ts` NEW
AI-11	◍	**ML Data Collection Worker** — triggers after COMPLETED/FAILED; logs full RideContext, all provider scores at booking time, selected provider, orchestration strategy, actual outcome, `timeToConfirm`, `timeToComplete`	`backend/src/workers/ml-data-collection.worker.ts` NEW

#	Priority	Task	Files
AI-12	⬤	**Prisma migration — cache + ML tables** — new `ProviderMetricsCache` (providerId, metricType, metricKey, metricValue, computedAt, unique on providerId+type+key) + `MLTrainingData` (contextSnapshot, providerScores, selectedProviderId, orchestrationStrategy, actualOutcome, timeToConfirm)	`backend/prisma/schema.prisma`
AI-13	⬤	**Frontend AI-enhanced results screen** — reliability score progress bar (green >90 / yellow >75 / red <75); AI reasoning text below each quote card; colored badges (MOST_RELIABLE / CHEAPEST / PREMIUM); Socket.IO events: `booking:ai_decision`, `booking:provider_queried`, `booking:failure_risk_update`; live status text: "Analyzing providers..." → "Querying Fast Track..." → "Confirmed!"	`frontend/src/App.jsx` MODIFY

AI Week 3 — Demand Forecasting + Fair Dynamic Pricing ☐ (0/5)

#	Priority	Task	Files
AI-14	⬤	**Demand Forecaster Service** — pattern matching (no ML yet); per zone/hour: avg rides last 4 weeks same day/hour, trend, day-of-week multiplier, Chennai events CSV multiplier, weather multiplier (rain +30%); outputs `predictedRides` + `confidenceLow/High` interval	`backend/src/services/ai/demand-forecaster.service.ts` NEW
AI-15	⬤	**Prisma migration — demand forecasts** — new `DemandForecast` (zone, forecastHour DateTime, predictedRides Float, confidenceLow Float, confidenceHigh Float, actualRides Int?, forecastAccuracy Float?, unique on zone+forecastHour)	`backend/prisma/schema.prisma`
AI-16	⬤	**Proactive driver positioning job** — nightly: generate 24h forecasts; detect supply gaps (predictedRides > available drivers in zone); trigger WhatsApp/SMS to drivers in shortage zone ("High demand 6–9 AM Airport. ₹100 bonus per ride"); track effectiveness in DB	`backend/src/jobs/demand-forecast-update.job.ts` NEW

#	Priority	Task	Files
AI-17	⚪	**Enhanced dynamic pricing in fare.service.ts** — add demand multiplier (+5–15% when demand > supply); weather: rain +8% / heavy rain +15% / storm +20%; traffic: low –5% / high +8% / jam +12%; time-of-day: late-night +10% / peak +5%; **hard cap: base fare × 1.2 max (+20% ceiling)**; Movzz Pass users always get base fare	`backend/src/services/fare.service.ts` ENHANCE
AI-18	⚪	**Pricing breakdown API + frontend** — quotes response adds `{ fareEstimate, baseFare, breakdown: [{factor, amount}], explanation }`; tap quote card → "Why this price?" modal showing each factor	`backend/src/controllers/booking.controller.ts` MODIFY, `frontend/src/App.jsx` MODIFY

AI Week 4 — Provider Analytics + User Personalization ☐ (0/5)

#	Priority	Task	Files
AI-19	⚪	**Provider Analytics Service** — mine acceptance patterns; behavioral profiles: airport specialist, short-ride vs long-distance, morning person vs night owl, price-sensitive vs volume-focused; outputs human-readable recommendations ("Send them more airport rides 6–9 AM")	`backend/src/services/ai/provider-analytics.service.ts` NEW
AI-20	⚪	**Provider Dashboard Endpoint** — `GET /api/v1/providers/:id/insights`; response: `overallStats`, `yourBestTimes[]`, `yourPreferredZones[]`, `earningsOptimization { recommendation, potential }`, `aiInsights[]`	`backend/src/controllers/provider-dashboard.controller.ts` NEW
AI-21	⚪	**User Personalization Service** — `buildUserProfile(userId)`: favorite providers (repeat 5-star), commute detection (same route weekly), pre-book vs last-minute, price sensitivity; `GET /api/v1/users/me/recommendations`: pre-filled route, suggested provider, Movzz Pass upsell if 12+ rides/month	`backend/src/services/ai/personalization.service.ts` NEW

#	Priority	Task	Files
AI-22	⚫	**Prisma migration — personalization tables** — new `UserPreference` (userId, preferenceType, preferenceKey, preferenceValue, confidence Float, learnedAt, lastUsedAt, unique on userId+type+key) + `UserPattern` (userId, patternType, patternData Json, confidence Float, detectedAt, lastOccurrence)	`backend/prisma/schema.prisma`
AI-23	⚫	**Admin AI Dashboard** — new endpoints: `GET /admin/ai-performance` (orchestration strategy distribution, prediction accuracy, success rate with vs without AI) + `GET /admin/demand-insights` (forecast vs actual, supply shortage alerts, zone-by-hour heatmap) + `GET /admin/provider-rankings`; Admin panel new AI Performance tab with charts	`backend/src/controllers/admin.controller.ts` ENHANCE, `frontend/src/Admin.jsx` MODIFY

SECTION 4 — Production & Deployment ☐ (0/14 — All New)

> Required before any real users. Run after Section 0 security fixes are complete.

Production Setup

#	Priority	Task	Notes
D1	⚫	**Railway project setup** — create Postgres + Redis + Node.js service; connect GitHub repo; auto-deploy on push	railway.app — ~₹3,000–5,000/month
D2	⚫	**Frontend deploy to Vercel** — import repo; set `VITE_API_URL` to Railway backend URL; SSL auto-configured	Auto-deploys on every git push to main
D3	⚫	**Domain setup** — buy movzz.in; point DNS to Railway/Vercel; SSL configured automatically	₹500–1,000/year
D4	⚫	**Production DB migration** — run `npx prisma migrate deploy` on Railway; seed initial data: Chennai zones, transport modes, base fare rates, provider list	Railway console
D5	⚫	**Set all production env vars** — JWT_SECRET (64-char random), Razorpay live keys, Twilio, Resend, Mapbox, Sentry DSNs, Google OAuth callback URL	Railway dashboard env vars
D6	⚫	**Razorpay production activation** — switch test → live keys; complete KYC verification (2–3 business days); set webhook `https://movzz.in/api/v1/payments/webhook`; test ₹1 live transaction	Razorpay dashboard
D7	⚫	**Seed first admin user** — `INSERT INTO "User" (phone, role) VALUES ('+91...', 'admin')` via Prisma Studio	Production DB

Testing

#	Priority	Task	Notes
D8	🟤	**Load testing (k6)** — ramp 10 → 50 → 100 concurrent users; target: 95% requests < 2s, 0% error, CPU < 70%, DB connections < 8	`backend/tests/load-test.js` NEW
D9	🟤	**End-to-end user testing** — 5–10 testers; flow: signup → OTP → quote → booking → SMS → status update; collect and fix feedback	Manual test plan
D10	🟤	**Mobile responsiveness audit** — iPhone Safari + Android Chrome; min 44×44px touch targets; no horizontal scroll; map controls visible	Chrome DevTools + real devices

Legal & Documentation

#	Priority	Task	Notes
D11	🟤	**Terms of Service** — use termly.io template; customize for ride-booking and reliability guarantee	`frontend/src/pages/Terms.jsx` NEW
D12	🟤	**Privacy Policy** — termly.io template; DPDP Act (India) compliance basics	`frontend/src/pages/Privacy.jsx` NEW
D13	🟤	**User FAQ / Help Center** — 4 sections: Getting Started, Pricing, Reliability, Account	`frontend/src/pages/Help.jsx` NEW
D14	🟤	**Env documentation** — fully document all env vars in both `.env.example` files; update README with local setup steps	`backend/.env.example`, `frontend/.env.example`, `README.md`

SECTION 5 — Execution Order Summary

```
IMMEDIATE (this week):
  S1 S2 S3 S4 S5 S6          ← 🔴 Critical security (JWT, race conditions, admin auth, env)
  S7 S8 S9 S10 S11           ← 🟠 High security (rate limit, socket leak, dual fare, CORS)

NEXT (before beta):
  #43                        ← 🟠 Real Twilio OTP (users cannot sign up without it)
  AI-9 AI-10 AI-11 AI-12 AI-13 ← 🟠 AI Week 2 (makes AI visible in UI)
  P0 #47 #48                 ← 🟠 Provider interface + first 2 real providers
  P1                         ← 🟠 Provider health dashboard
  D1 D2 D3 D4 D5             ← 🔴 Deploy to production
  D6 D7 D8 D9 D10            ← 🟠 Activate services + test
  D11 D12                    ← 🟠 Legal (Terms + Privacy)

POST-LAUNCH:
  #49 #50                    ← 🟡 Uber/Ola/Rapido + provider onboarding
  #45 #46                    ← 🟡 WhatsApp + CDN
  AI-14 AI-15 AI-16 AI-17 AI-18 ← 🟡 AI Week 3 (demand forecasting + dynamic pricing)
  D13 D14                    ← 🟡 FAQ + env docs
  AI-19 AI-20 AI-21 AI-22 AI-23 ← 🟤 AI Week 4 (analytics + personalization)
  #44 #51 #52 #53 #54 #55    ← 🟤 FCM + React Native mobile apps
```

AI Success Metrics (Track Weekly After AI Week 1)

Metric	Baseline	Target (Post-AI)
Booking success rate	~75%	92%+
Avg time to confirmation	~45 seconds	<20 seconds
Rides needing manual ops	~20%	<5%
AI prediction accuracy	—	>90% (predicted vs actual outcome)
Revenue from dynamic pricing	0%	+10–15%
User satisfaction (avg rating)	~4.2	4.7+

---

*MOVZZ Master Roadmap · March 2026 · **50/107 tasks done (47%)***